МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 2

Ветвления и циклы в Python

Вариант 13

по дисциплине «Программирование на языке Python»

Выполнил
Тебаев Мансур Дахирович
Группа ЭНЭ-б-о-25-2
Проверил _____

Ставрополь 2026

1 Тема и цель работы

Тема — Ветвления и циклы в Python. Освоить условные операторы if и match, циклы for и while, управление повторением и проверку численных алгоритмов с оценкой остатка ряда.

Во всех обязательных группах индивидуальных заданий выбран вариант 13. Количество групп: 4.

Локальный проект: python-lab-02. Подготовлен локальный Git репозиторий; удалённый адрес будет добавлен после публикации владельцем. Ссылка на удалённый репозиторий: не опубликован.

2 Среда и порядок запуска

Проверка выполнена в Windows 10, Python 3.14.6, с использованием виртуального окружения. Сторонние библиотеки для учебных программ не требуются. Числа вводятся в десятичной записи с точкой; если иное не указано, поля вводятся отдельно, каждое с новой строки. Для списков вводится одна строка чисел через пробел.

Запуск и пошаговая отладка выполнены в VS Code 1.144.0 с Python Debugger 2026.6.0 через Extension Host API. Контрольные входы поданы из подготовленных данных. Отладчик подтвердил значения переменных и следующий шаг; журнал сохранён в results/vscode_debug.json.

Из папки проекта для создания собственного окружения и запуска используются команды:

```
python -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe branch.py
.\.venv\Scripts\python.exe verify.py
```

Файл verify.py использует тот интерпретатор, которым он запущен. Эталонные данные лежат в fixtures.json, полные результаты — в results/checks.json и results/checks.txt. Код 0 ожидается для корректного ввода, код 1 — для проверяемых ошибок диапазона или формата. Импорт учебных модулей не запускает диалог ввода благодаря защите __name__.

3 Разобранные примеры

Кусочная функция

Ввести конечное вещественное x от −100 до 100. Вычислить $y = 2x^2 + \cos x$ при $x \le 0$; $y = x + 1$ при $0 < x < 5$; $y = \sin(2x) - x^2$ при $x \ge 5$. Углы заданы в радианах.

В цепочке if–elif–else выполняется ровно одна ветвь. После проверки $x \le 0$ условие $x < 5$ уже означает $0 < x < 5$. Значения 0 и 5 относятся к первой и третьей ветвям соответственно.

Файл полного решения: python-lab-02/branch.py.

```
import math
import sys


def main():
    x = float(input())
    if not -100 <= x <= 100:
        print('Ошибка: x вне [-100, 100]', file=sys.stderr)
        return 1
    if x <= 0:
        y = 2 * x ** 2 + math.cos(x)
    elif x < 5:
        y = x + 1
    else:
        y = math.sin(2 * x) - x ** 2
    print(f'y = {y:.6f}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0	y = 1.000000	y = 1.000000
-2	y = 7.583853	y = 7.583853
2	y = 3.000000	y = 3.000000
5	y = -25.544021	y = -25.544021
-1e-06	y = 1.000000	y = 1.000000
1e-06	y = 1.000001	y = 1.000001
4.999999	y = 5.999999	y = 5.999999
5.000001	y = -25.544033	y = -25.544033
-100	y = 20000.862319	y = 20000.862319
100	y = -10000.873297	y = -10000.873297
101	Ошибка: x вне [-100, 100] Код 1	Ошибка: x вне [-100, 100] Код 1

Сезон через if

По целому номеру месяца определить сезон. Для числа вне 1–12 вывести сообщение об ошибке.

Проверка принадлежности кортежу объединяет месяцы одного сезона. else обрабатывает недопустимые номера.

Файл полного решения: python-lab-02/season_if.py.

```
def main():
    month = int(input())
    if month in (12, 1, 2):
        season = 'Зима'
    elif month in (3, 4, 5):
        season = 'Весна'
    elif month in (6, 7, 8):
        season = 'Лето'
```

```
    elif month in (9, 10, 11):
        season = 'Осень'
    else:
        season = 'Ошибка: номер месяца'
    print(season)


if __name__ == '__main__':
    main()
```

Месяц	Ожидается	if факт	match факт
0	Ошибка: номер месяца	Ошибка: номер месяца	Ошибка: номер месяца
1	Зима	Зима	Зима
2	Зима	Зима	Зима
3	Весна	Весна	Весна
4	Весна	Весна	Весна
5	Весна	Весна	Весна
6	Лето	Лето	Лето
7	Лето	Лето	Лето
8	Лето	Лето	Лето
9	Осень	Осень	Осень
10	Осень	Осень	Осень
11	Осень	Осень	Осень
12	Зима	Зима	Зима
13	Ошибка: номер месяца	Ошибка: номер месяца	Ошибка: номер месяца

Сезон через match

Решить ту же задачу с помощью match и литеральных образцов.

Образцы объединены символом |. case _ используется последним для остальных значений. Обе реализации имеют одинаковый вывод; match непосредственно перечисляет образцы, if проверяет логические выражения.

Файл полного решения: python-lab-02/season_match.py.

```
def main():
    month = int(input())
    match month:
        case 12 | 1 | 2:
            season = 'Зима'
        case 3 | 4 | 5:
            season = 'Весна'
        case 6 | 7 | 8:
            season = 'Лето'
        case 9 | 10 | 11:
            season = 'Осень'
        case _:
            season = 'Ошибка: номер месяца'
    print(season)


if __name__ == '__main__':
    main()
```

Конечная сумма

Ввести целое n от 0 до 10000 и вещественное $0 < x \leq 100$. Вычислить $S = \Sigma \ln(kx)/k^2$ для k от 1 до n.

Накопитель начинается с 0.0. range(1, n + 1) включает n. При n = 0 цикл не выполняется и сумма равна нулю. Область $x > 0$ обеспечивает определённость логарифма.

Файл полного решения: python-lab-02/finite_sum.py.

```
import math
import sys


def main():
```

```
    n = int(input())
    x = float(input())
    if not (0 <= n <= 10000 and 0 < x <= 100):
        print('Ошибка: недопустимые n или x', file=sys.stderr)
        return 1
    total = 0.0
    for k in range(1, n + 1):
        total += math.log(k * x) / k ** 2
    print(f'S = {total:.10f}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0; 1	S = 0.0000000000	S = 0.0000000000
1; 1	S = 0.0000000000	S = 0.0000000000
2; 1	S = 0.1732867951	S = 0.1732867951
3; 2	S = 1.2388051563	S = 1.2388051563
1; 0	Ошибка: недопустимые n или x Код 1	Ошибка: недопустимые n или x Код 1

Сумма до сигнального нуля

Последовательно вводить целые числа до первого нуля. Суммировать только положительные; отрицательные пропускать.

break завершает цикл до прибавления нуля, continue передаёт управление следующему вводу при отрицательном числе. Накопитель сохраняет сумму ранее введённых положительных чисел.

Файл полного решения: python-lab-02/positive_sum.py.

```
def main():
    total = 0
    while True:
        number = int(input())
        if number == 0:
            break
        if number < 0:
            continue
        total += number
    print(f'Сумма: {total}')


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
3; -2; 5; 0	Сумма: 8	Сумма: 8
0	Сумма: 0	Сумма: 0
-9; -1; 0	Сумма: 0	Сумма: 0
1; 2; 3; 0	Сумма: 6	Сумма: 6

Итерационное извлечение корня

Для a от 0 до 1000000 вычислить квадратный корень итерациями x_next = (x + a/x)/2 с точностью изменения 10^{-10} и ограничением 1000 итераций. Сравнить с math.sqrt.

Для a = 0 возвращается ноль без деления. Начальное x = max(1, a) положительно. Условие остановки ограничивает изменение двух приближений; фактическая ошибка дополнительно оценивается сравнением с math.sqrt.

Файл полного решения: python-lab-02/sqrt_iteration.py.

```
import math
import sys


def main():
    a = float(input())
    if not 0 <= a <= 1e6:
        print('Ошибка: a вне [0, 1000000]', file=sys.stderr)
        return 1
    x = max(1.0, a)
    iterations = 0
    if a == 0:
        x = 0.0
    else:
        while iterations < 1000:
            previous = x
            x = (previous + a / previous) / 2
            iterations += 1
            if abs(x - previous) <= 1e-10:
                break
        else:
            print('Ошибка: предел итераций', file=sys.stderr)
            return 1
    print(f'Корень: {x:.12f}')
    print(f'Итераций: {iterations}')
    print(f'Расхождение: {abs(x - math.sqrt(a)):.3e}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0	Корень: 0.000000000000	Корень: 0.000000000000

Ввод или сценарий	Ожидается	Получено
1	Корень: 1.000000000000	Корень: 1.000000000000
2	Корень: 1.414213562373	Корень: 1.414213562373
0.0001	Корень: 0.010000000000	Корень: 0.010000000000
1000000.0	Корень: 1000.000000000000	Корень: 1000.000000000000
-1	Ошибка: a вне [0, 1000000] Код 1	Ошибка: a вне [0, 1000000] Код 1

Экспоненциальный интеграл

Для $0{,}1 \le x \le 1$ вычислить $Ei(x) = \gamma + \ln x + \Sigma\, x^k/(k \cdot k!)$, $k \ge 1$, где $\gamma = 0{,}5772156649015329$. Точность оценки остатка — 10^{-10}.

Первый член $a_1 = x$. Деление a_{k+1}/a_k даёт $xk/(k + 1)^2$. На заданном отрезке отношение не больше 1/4. Поэтому остаток после последнего включённого члена не больше $|a_k|/3$. При $x = 1$ первые три члена: 1; 1/4; 1/18, их сумма 1,3055555556.

Файл полного решения: python-lab-02/exponential_integral.py.

```
import math
import sys


def main():
    x = float(input())
    if not 0.1 <= x <= 1:
        print('Ошибка: x вне [0.1, 1]', file=sys.stderr)
        return 1
    eps = 1e-10
    term = x
    total = term
    k = 1
    while abs(term) / 3 > eps:
        if k >= 1000:
```

```
            print('Ошибка: предел членов', file=sys.stderr)
            return 1
        term *= x * k / (k + 1) ** 2
        k += 1
        total += term
    result = 0.5772156649015329 + math.log(x) + total
    print(f'Ei = {result:.10f}')
    print(f'Членов: {k}')
    print(f'Оценка остатка: {abs(term) / 3:.3e}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0.1	Ei = -1.6228128140 Допуск $1{,}5 \cdot 10^{-10}$	Ei = -1.6228128140
0.5	Ei = 0.4542199049 Допуск $1{,}5 \cdot 10^{-10}$	Ei = 0.4542199049
1	Ei = 1.8951178164 Допуск $1{,}5 \cdot 10^{-10}$	Ei = 1.8951178163
0	Ошибка: x вне [0.1, 1] Код 1	Ошибка: x вне [0.1, 1] Код 1

4 Индивидуальные задания

Индивидуальное задание 1 вариант 13

Сборщику помидоров за каждый килограмм платят 0,30 руб. при сборе до 50 кг включительно; 0,50 руб. при сборе свыше 50 до 75 кг включительно; 0,65 руб. при сборе свыше 75 до 90 кг включительно; 0,70 руб. при сборе свыше 90 кг и дополнительно премию 20 руб. Ввести массу x от 0 до 1000 кг и определить оплату.

Тариф соответствующего диапазона применяется ко всей массе. Это не накопительный расчёт по ступеням. Премия прибавляется один раз только при x > 90. Например, при 90 кг оплата 58,50 руб., при 100 кг — 90,00 руб.

Файл полного решения: python-lab-02/individual_01.py.

```
import sys


def main():
    x = float(input())
    if not 0 <= x <= 1000:
        print('Ошибка: масса вне [0, 1000]', file=sys.stderr)
        return 1
    if x <= 50:
        payment = 0.30 * x
    elif x <= 75:
        payment = 0.50 * x
    elif x <= 90:
        payment = 0.65 * x
    else:
        payment = 0.70 * x + 20
    print(f'Оплата: {payment:.2f} руб.')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0	Оплата: 0.00 руб.	Оплата: 0.00 руб.
50	Оплата: 15.00 руб.	Оплата: 15.00 руб.
50.001	Оплата: 25.00 руб.	Оплата: 25.00 руб.
75	Оплата: 37.50 руб.	Оплата: 37.50 руб.
75.001	Оплата: 48.75 руб.	Оплата: 48.75 руб.
90	Оплата: 58.50 руб.	Оплата: 58.50 руб.
90.001	Оплата: 83.00 руб.	Оплата: 83.00 руб.
1000	Оплата: 720.00 руб.	Оплата: 720.00 руб.
-.1	Ошибка: масса вне [0, 1000] Код 1	Ошибка: масса вне [0, 1000] Код 1

Индивидуальное задание 2 вариант 13

Ввести два натуральных числа x и y. Определить, делится ли x на y без остатка.

При x % y == 0 деление является нацело. Проверка натуральности исключает нулевой делитель. Для x < y остаток равен x, поэтому ответ отрицательный.

Файл полного решения: python-lab-02/individual_02.py.

```
import sys


def main():
    x = int(input())
    y = int(input())
    if x < 1 or y < 1:
        print('Ошибка: нужны натуральные числа', file=sys.stderr)
```

```
        return 1
    if x % y == 0:
        print('Делится')
    else:
        print('Не делится')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
12; 3	Делится	Делится
13; 3	Не делится	Не делится
1; 1	Делится	Делится
2; 5	Не делится	Не делится
1; 0	Ошибка: нужны натуральные числа Код 1	Ошибка: нужны натуральные числа Код 1

Индивидуальное задание 3 вариант 13

Ввести натуральное n, $1 \le n \le 100000$. Найти и вывести все натуральные делители числа.

В цикле перебираются k от 1 до n включительно; k добавляется при n % k == 0. Делители выводятся по возрастанию. Цикл конечен, поскольку range содержит ровно n значений. Сложность — O(n) по времени, память зависит от числа делителей.

Файл полного решения: python-lab-02/individual_03.py.

```
import sys
```

```
def main():
    n = int(input())
    if not 1 <= n <= 100000:
        print('Ошибка: n вне [1, 100000]', file=sys.stderr)
        return 1
    divisors = []
    for k in range(1, n + 1):
        if n % k == 0:
            divisors.append(k)
    print('Делители:', *divisors)
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
1	Делители: 1	Делители: 1
13	Делители: 1 13	Делители: 1 13
12	Делители: 1 2 3 4 6 12	Делители: 1 2 3 4 6 12
6	Делители: 1 2 3 6	Делители: 1 2 3 6
100000	36 делителей от 1 до 100000	36 делителей от 1 до 100000
0	Ошибка: n вне [1, 100000] Код 1	Ошибка: n вне [1, 100000] Код 1

Трассировка поиска делителей

k	Остаток 6 % k	Делители после шага
1	0	[1]
2	0	[1, 2]

k	Остаток 6 % k	Делители после шага
3	0	[1, 2, 3]
4	2	[1, 2, 3]
5	1	[1, 2, 3]
6	0	[1, 2, 3, 6]

Остановка перед проверкой n % k == 0 при n = 6, k = 3 показывает divisors = [1, 2]. Следующий шаг даёт нулевой остаток и добавляет 3. Таблица получена при трассировке Python; этот же шаг можно повторить в графическом отладчике.

Индивидуальное задание 4 вариант 13

Вычислить $\mathrm{sh}(x) = (e^x - e^{-x})/2 = \Sigma\, x^{2k+1}/(2k+1)!$, $k \geq 0$, на отрезке $[-1; 1]$ с оценкой остатка не более 10^{-10}. Использовать рекуррентное получение членов, ограничить их число 1000 и выполнить независимую проверку.

Первый член $a_0 = x$. При делении соседних членов степени сокращаются, а $(2k+3)!/(2k+1)! = (2k+2)(2k+3)$. Поэтому $a_{k+1} = a_k \cdot x^2/((2k+2)(2k+3))$. Для $|x| \leq 1$ отношение модулей не больше $1/6 < 1/2$. Сумма модулей отброшенных членов меньше $|a_k|$; достаточно включать члены до $|a_k| \leq 10^{-10}$. Это оценка усечения ряда, отдельно от округления float. При x = 0 включается один нулевой член. Проверка использует прямые степени и факториалы 50 членов, затем math.sinh.

Файл полного решения: python-lab-02/individual_04.py.

```
import math
import sys


def main():
    x = float(input())
    if not -1 <= x <= 1:
        print('Ошибка: x вне [-1, 1]', file=sys.stderr)
        return 1
```

```
    eps = 1e-10
    k = 0
    term = x
    total = term
    while abs(term) > eps:
        if k + 1 >= 1000:
            print('Ошибка: предел членов', file=sys.stderr)
            return 1
        term *= x * x / ((2 * k + 2) * (2 * k + 3))
        k += 1
        total += term
    reference = math.fsum(
        x ** (2 * j + 1) / math.factorial(2 * j + 1)
        for j in range(50)
    )
    print(f'sinh = {total:.10f}')
    print(f'Членов: {k + 1}')
    print(f'Оценка остатка: {abs(term):.3e}')
    print(f'Проверка 50 членов: {reference:.12f}')
    print(f'Расхождение: {abs(total - reference):.3e}')
    print(f'Расхождение с math.sinh: {abs(total - math.sinh(x)):.3e}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
-1	sinh = -1.1752011936	sinh = -1.1752011936
-0.5	sinh = -0.5210953055	sinh = -0.5210953055
0	sinh = 0.0000000000	sinh = 0.0000000000
0.5	sinh = 0.5210953055	sinh = 0.5210953055

Ввод или сценарий	Ожидается	Получено
1	sinh = 1.1752011936	sinh = 1.1752011936
-1.0001	Ошибка: x вне [-1, 1] Код 1	Ошибка: x вне [-1, 1] Код 1
1.0001	Ошибка: x вне [-1, 1] Код 1	Ошибка: x вне [-1, 1] Код 1

Трассировка суммирования ряда

k	Последний член	Частичная сумма
0	1	1
1	0.166666666667	1.16666666667
2	0.00833333333333	1.175
3	0.000198412698413	1.1751984127
4	2.7557319224e-06	1.17520116843
5	2.50521083854e-08	1.17520119348
6	1.60590438368e-10	1.17520119364
7	7.64716373182e-13	1.17520119364

При x = 1 первые три включённых члена: 1; 1/6; 1/120. Частичная сумма равна 1,175. Всего включено 8 членов, последний равен приблизительно $7{,}64716 \cdot 10^{-13}$. Оценка остатка меньше 10^{-10}; проверка 50 прямых членов даёт 1,175201193644. Полный вывод контрольного запуска:

```
sinh = 1.1752011936
Членов: 8
Оценка остатка: 7.647e-13
Проверка 50 членов: 1.175201193644
Расхождение: 2.665e-15
Расхождение с math.sinh: 2.665e-15
```

Для воспроизведения остановки поставить точку на term *= ...: перед первым выполнением k = 0, term = 1, total = 1. После шага term = 1/6, затем k увеличивается до 1, после прибавления total = 7/6. Ограничение 1000 предотвращает бесконечное повторение при изменении условий алгоритма.

5 Трассировка и воспроизведение в редакторе

Трассировочные таблицы обязательных заданий 3 и 4 приведены после их проверок. Дополнительно выполнены остановки и пошаговое выполнение в отладчике VS Code; значения согласуются с расчётами.

В VS Code выполнены остановки в individual_03.py на строке 10 при k = 3 и в individual_04.py на строке 17 при k = 0. Шаг в первом случае перешёл к добавлению делителя; во втором изменил term с 1 до 1/6. Итоговые результаты совпали с расчётом. Все проверки verify.py также успешно выполнены запуском из VS Code. Журнал остановок и вывода сохранён в results/vscode_debug.json.

6 Ответы на вопросы для защиты

1. Присваивание и сравнение. Знак = связывает имя со значением; == сравнивает значения и даёт bool.

2. Ложные значения. Ложны False, None, числовой ноль и пустые контейнеры. Строка "0" непустая, поэтому истинна.

3. Логические операции. not возвращает bool. and и or вычисляют второй операнд только при необходимости и возвращают один из операндов.

4. Несколько if и цепочка. Независимые if могут выполнить несколько блоков. В if–elif–else выбирается только первая истинная ветвь.

5. Порядок условий. При пересекающихся диапазонах более ранняя истинная проверка исключает последующие; границы следует распределять явно.

6. Условное выражение. a if condition else b выбирает одно значение. Оно удобно для короткого присваивания без сложных действий в ветвях.

7. Версия match. Сопоставление match доступно начиная с Python 3.10.

8. Числовой и строковый образцы. Литерал 1 сопоставляется с числом, "1" — со строкой; input без int возвращает строку.

9. Образец по умолчанию. case _ совпадает с любым значением и ничего не связывает. Его ставят последним, чтобы не перекрыть остальные случаи.

10. Объединение образцов. Запись case 1 | 2 | 3 объединяет альтернативные образцы одной ветви.

11. Завершение case. break после case не нужен; сквозного перехода нет. Без совпадений и case _ блок match не выполняет ни одной ветви.

12. Захват имени. case name связывает имя с сопоставляемым значением, а не сравнивает с прежним значением переменной. Для сравнения используют подходящий литерал, квалифицированную константу или guard.

13. Выбор if или match. if удобен для диапазонов и произвольных условий; простой match — для перечисления конкретных команд и значений.

14. for и range. for последовательно получает элементы итерируемого объекта. range описывает арифметическую последовательность целых, исключая stop, и не создаёт список всех чисел.

15. Нечётные числа по убыванию. range(15, 0, −2) даёт 15, 13, …, 1. В коде используется обычный минус: range(15, 0, -2). Граница 0 не включается.

16. Нулевое число итераций while. Если условие ложно при первой проверке, тело while не выполняется.

17. break и continue. break завершает ближайший содержащий его цикл, continue начинает его следующую итерацию. Внешние циклы непосредственно не завершаются.

18. Опасность continue. Если continue пропускает изменение переменной условия while, условие может навсегда оставаться истинным.

19. else у цикла. Блок else выполняется при нормальном окончании цикла, в том числе без итераций, но пропускается при break.

20. Начальные накопители. Нейтральный элемент суммы — 0, произведения — 1. Для поиска экстремума непустой последовательности удобно взять первый элемент.

21. Остановка и ошибка. Малое изменение приближений само по себе не гарантирует близость к точному ответу. Оценка остатка ряда даёт отдельную границу ошибки усечения при доказанных условиях.

22. Сравнение float. math.isclose использует относительный и абсолютный допуски. Для чисел около нуля необходим осмысленный abs_tol.

23. Пограничные тесты. Проверяют саму границу и значения с обеих сторон, а также концы допустимого диапазона и недопустимый ввод.

24. Трассировочная таблица. Таблица фиксирует изменения счётчиков, условий и накопителей по шагам, поэтому показывает причину результата, которую не видно в конечной строке.

25. Потоки и код завершения. stdout содержит результат, stderr — диагностику; 0 означает успешное завершение, ненулевой код — ошибку. print в stderr только пишет сообщение; остановку выполняют return из main с sys.exit либо исключение SystemExit.

7 Вывод

Разработаны и проверены 11 учебных программ. Успешно пройдены все 85 проверок, включая предусмотренные ошибочные входы. Подтверждены граничные условия, порядок обработки данных и соответствие фактических результатов ожидаемым. Освоены ветвления, конечные циклы и суммирование ряда по рекуррентному соотношению с обоснованной оценкой остатка.

8 Источники

Методические материалы курса. Лабораторная работа № 2. Ветвления и циклы в Python. Общая папка: https://disk.yandex.ru/d/yT9RQ8LG9aQ_UA

Python 3.14. Стандартные типы и управление потоком выполнения: https://docs.python.org/3.14/library/stdtypes.html и https://docs.python.org/3.14/tutorial/controlflow.html

Python 3.14. Модуль math: https://docs.python.org/3.14/library/math.html